

$e^{-(rows-1)}$

$$2 * rows - i + 1$$

3-1 i

3-2

3-3

	0	1	2	3	4
0			*		..
1			*	-	*
2			*	-	*
3			*	-	*
4				*	

rows

1

2

```
for(int i=0; i<2*row-1; i++){
```

rows

```
for(int j=0; j<2*rows-1; j++) {
```

rows - 1

```
for (i = 0; i < rows; i++) {
```

if ($j < \text{rows} - i - 1$)

```
printf(" ");
```

```
else if (j < rows + i) {
```

```
if((i+j)%2 == 0)
```

```
printf("*");
```

else

```
printf("-");
```

3

else {

if ($j < i - \text{rows} + 1$)

```
printf(" ");
```

```

else if (j < (2 * rows - 1) - (i - rows + 1)) {
    // if i is at the bottom edge, then j is at the top edge
    // and vice versa
    // so we need to swap them
    swap(arr[i][j], arr[i - rows + 1][j]);
}

```

if $((i+j) \% 2 == 0)$

```
printf("*");
```

else

```
printf("-");
```

3

3

3

```
printf("\n");
```

3

$$j \leq \text{rows} + i - 1$$

first 3 rows

0 1 2 3 4 5 6



1 * - *

2 * - * - *

3 * - * - * - *

4. * - * - *

$$5 \quad * \quad - \quad * \quad \bullet$$

6 *.

rows = 4

break, continue, goto, exit.

→ int i;

```
for(i=0 ; i<10 ; i++) {  
    if(i>5)  
        break;  
    printf("%d", i);  
}
```

→ printf("%d", i);

O/p

0 1 2 3 4 5 6

→ break — switch, or loop.

```
int i, j;  
for(i=0; i<10; i++){  
    for(j=0; j<10; j++){  
        printf("%d", i+j);  
        if(i+j>9) ✓  
            break;  
    }  
    printf("\n");  
}
```

O/p

<u>j</u>	<u>0</u>	1	2	3	4	5	6	7	8	9
0	0	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9	10
2	2	3	4	5	6	7	8	9	10	
3	3	4	5	6	7	8	9	10		
4	4	5	6	7	8	9	10			
5	5	6	7	8	9	10				
6										
7										
8										
9										

```
for(int i=0; i<10; i++){  
    if(i%2 == 0)  
        continue; ✓  
    printf("%d", 2*i);  
}
```

O/p

2 6 10 14 18

✓✓

→ continues the next iteration
→ loop. ✓

```
int i=0;
```

```
while(i < 10) {  
    printf("%d", i);  
    if(i%2 == 0)  
        continue;  
    i++;  
}
```

O/P

00000000000000
50000000000000
00000

Infinite times

```
int i=0;
```

```
while(i < 10) {  
    printf("%d", i);  
    if(i%2 == 1)  
        continue;  
    i++;  
}
```

O/P

01111111111111
11111

infinite times

```
int i=0;
```

```
while(i < 10) {  
    i++;  
    printf("%d", i);  
    if(i%2 == 1)  
        continue;  
}
```

O/P

1 2 3 4 5 6 7 8 9 10

Useless

↓
Because no
statements
after continue
in loop. }

★ one more way to
do the same pattern

$$\text{rows} = 2(i) \oplus 0 \quad 0 \quad 1 \quad 2 \quad 3 \quad 4$$

			*				1
1		*	-	*			3
2		*	-	*	-	*	5
	0		*	-	*		3
	1			*			1

```
for(int i=0; i<rows; i++){
    for(int j=0; j<rows-i-1; j++){
        printf(" ");
        for(int k=0; k<2*i+1; k++){
            if(k%2)
                printf("-");
            else
                printf("*");
        }
        printf("\n");
    }
}
```

$$(2 * \text{rows} - 1) - 2(i + 1)$$

$$5 - 2 = 3$$

$$5 - 4 = 1$$

```
for(int i=0; i<rows-1; i++){
    for(int j=0; j<i+1; j++){
        printf(" ");
        for(int k=0; k<((2*rows-1)-(2*(i+1))); k++){
            if(k%2)
                printf("-");
            else
                printf("*");
        }
        printf("\n");
    }
}
```